

Otimizador de Rotas Hospitalares e Assistente Logístico IA

1. Introdução

A logística de entregas hospitalares exige coordenação precisa entre veículos, prazos críticos, prioridades de atendimento e limites operacionais. Em cenários reais, a entrega de medicamentos, insumos e materiais essenciais precisa ser executada com rapidez e confiabilidade, pois atrasos podem impactar diretamente a qualidade do cuidado ao paciente. O problema central é, portanto, um caso de roteamento com múltiplas restrições, em que a solução precisa ser eficiente, viável e aplicável ao ambiente operacional.

2. Objetivo

Este projeto teve como objetivo desenvolver uma solução de roteamento para entregas hospitalares, a partir de uma base inicial de código para otimização de rotas, ampliando o modelo para considerar múltiplos veículos, prioridades de entrega, capacidade de carga, autonomia operacional e requisitos de distribuição equilibrada.

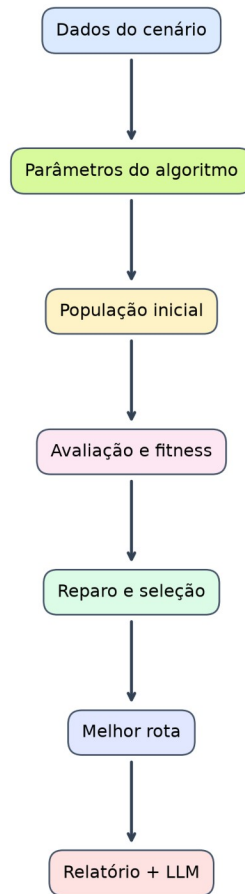
Além disso, o trabalho incluiu a integração do motor de otimização com modelos de linguagem para gerar instruções e relatórios em linguagem natural, tornando o resultado mais acessível para uso operacional.

3. Implementação do algoritmo genético

A base do desenvolvimento foi expandida do problema clássico do caixeiro viajante para um modelo de VRP hospitalar mais realista. O algoritmo genético foi implementado como mecanismo de busca evolutiva, com representação das rotas como sequências de entregas e cortes entre veículos.

A população inicial foi gerada a partir de permutações aleatórias, e cada solução foi avaliada por uma função de fitness que combina distância total e penalidades por

violações de restrição. Ao longo das gerações, operações de crossover, mutação e ajuste de rotas foram aplicadas para melhorar a qualidade das soluções. A estrutura do algoritmo foi construída para explorar combinações de entregas e distribuí-las entre veículos sem comprometer a viabilidade operacional.



4. Estratégias para restrições operacionais

Para tornar a solução aplicável ao cenário do projeto, foram adotadas estratégias específicas para lidar com as restrições adicionais do problema. Em conjunto, essas medidas permitiram que a otimização considerasse não apenas distâncias, mas também a viabilidade real da operação.

As entregas com maior prioridade foram avaliadas com penalização forte quando deslocadas para posições tardias, preservando a lógica de atendimento crítico. A capacidade de carga foi tratada como limite operacional, com penalidade para

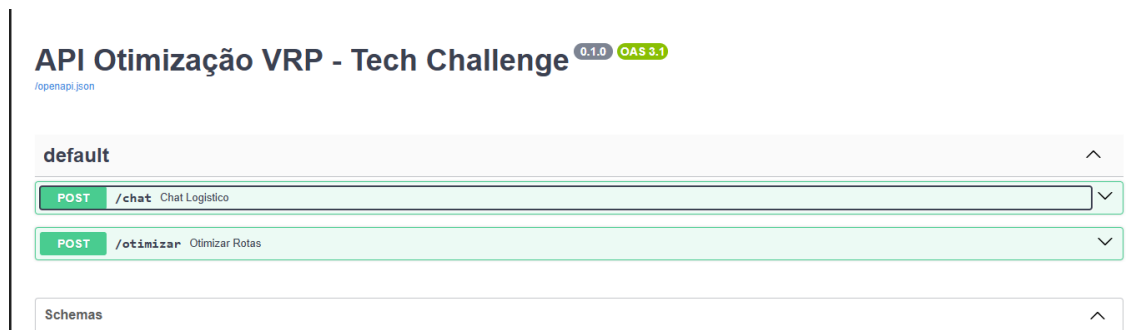
veículos que excedem a demanda suportada. A autonomia também foi incorporada à avaliação, evitando rotas que extrapolam o deslocamento operacional do veículo.

Enquanto que a divisão das entregas entre veículos foi modelada por pontos de corte e reparo de rotas, de forma a manter a solução coerente mesmo quando a geração inicial era incompatível com as regras do cenário.

5. Integração com LLMs e abordagem de prompts

A camada de inteligência artificial foi integrada ao backend em FastAPI e ao agente em LangChain, usando o modelo Gemini 3.5 Flash Lite. O fluxo principal consiste em gerar um resumo estruturado em JSON com métricas de fitness, distância total, penalidades, capacidade e autonomia por veículo, entregas críticas, recomendações e detalhes de rota. Esse payload é repassado ao modelo de linguagem como contexto operacional, e a IA transforma os dados em instruções claras para as equipes, relatórios resumidos e respostas em linguagem natural.

A abordagem adotada foi tratá-la como interpretadora do contexto, e não como origem da decisão logística. O prompt orienta o modelo a responder em português do Brasil, priorizar informações relevantes, resumir as métricas principais e destacar entregas críticas e gargalos operacionais.



Cenário e Parâmetros

Selecione o Cenário

base

Gerações

10

Tamanho da População

10

Taxa de Mutação

0.25

0.001.00

Executar Algoritmo Genético e IA

Deploy

5. [Normal] Entrega D18 (Demanda: 2) — Localização: (X: 650, Y: 360) | Dist. Acumulada: 926,42 km

6. [Normal] Entrega D19 (Demanda: 4) — Localização: (X: 720, Y: 175) | Dist. Acumulada: 1.124,22 km

7. [Normal] Entrega D08 (Demanda: 3) — Localização: (X: 615, Y: 125) | Dist. Acumulada: 1.240,52 km

8. [Baixa] Entrega D10 (Demanda: 2) — Localização: (X: 755, Y: 65) | Dist. Acumulada: 1.392,84 km (Retorno ao depósito em seguida).

3. Destaques Críticos e Recomendações Logísticas

Entregas Críticas: Foram mapeadas 3 entregas críticas no total (D01, D07 e D15). O algoritmo garantiu 100% de eficácia no atendimento prioritário, alocando todas elas obrigatoriamente como a primeira parada (posição 1) de cada veículo (V01, V02 e V03 respectivamente), obtendo 0 violações de prioridade crítica.

Recomendações da Análise:

1. Monitorar veículos próximos do limite de capacidade: O veículo V03 opera com 90,63% de sua capacidade ocupada e o veículo V02 com 87,50%. Recomenda-se cautela no manuseio das cargas para evitar estouros volumétricos de última hora.

2. Monitorar veículos próximos do limite de autonomia: O veículo V03 atingiu 98,48% de utilização de sua autonomia (percorrendo 1.772,64 km de um limite de 1.800 km). Qualquer desvio de rota, trânsito ou imprevisto colocará em risco o retorno deste veículo ao depósito central. Sugere-se avaliar um reforço ou rebalanceamento de carga para as próximas execuções.

Ver JSON Completo gerado pelo vrp_reporting.py

{

"report" : {

"schema_version" : "1.0"

"niransa" : f

Ex: Qual veículo percorre a maior distância?

Cenário e Parâmetros

Selecione o Cenário

base

Gerações

10

Tamanho da População

10

Taxa de Mutação

0.25

0.001.00

Executar Algoritmo Genético e IA

Deploy

3. Destaques Críticos e Recomendações Logísticas

Entregas Críticas: Foram mapeadas 3 entregas críticas no total (D01, D07 e D15). O algoritmo garantiu 100% de eficácia no atendimento prioritário, alocando todas elas obrigatoriamente como a primeira parada (posição 1) de cada veículo (V01, V02 e V03 respectivamente), obtendo 0 violações de prioridade crítica.

Recomendações da Análise:

1. Monitorar veículos próximos do limite de capacidade: O veículo V03 opera com 90,63% de sua capacidade ocupada e o veículo V02 com 87,50%. Recomenda-se cautela no manuseio das cargas para evitar estouros volumétricos de última hora.

2. Monitorar veículos próximos do limite de autonomia: O veículo V03 atingiu 98,48% de utilização de sua autonomia (percorrendo 1.772,64 km de um limite de 1.800 km). Qualquer desvio de rota, trânsito ou imprevisto colocará em risco o retorno deste veículo ao depósito central. Sugere-se avaliar um reforço ou rebalanceamento de carga para as próximas execuções.

Ver JSON Completo gerado pelo vrp_reporting.py

Tire dúvidas sobre as rotas

Qual veículo percorre a maior distância?

O veículo V03 percorre a maior distância, com um total de 1772,64 (unidades).

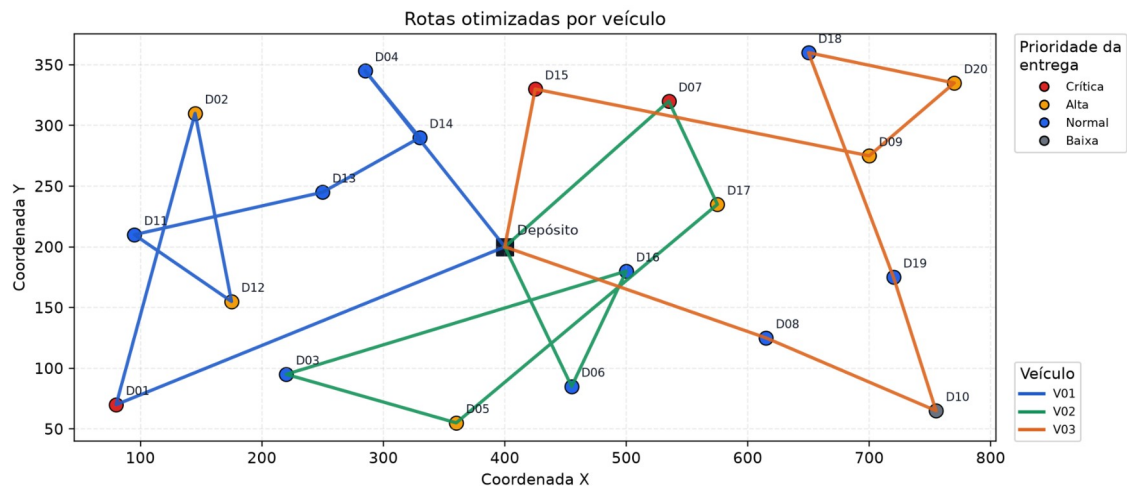
Ex: Qual veículo percorre a maior distância?

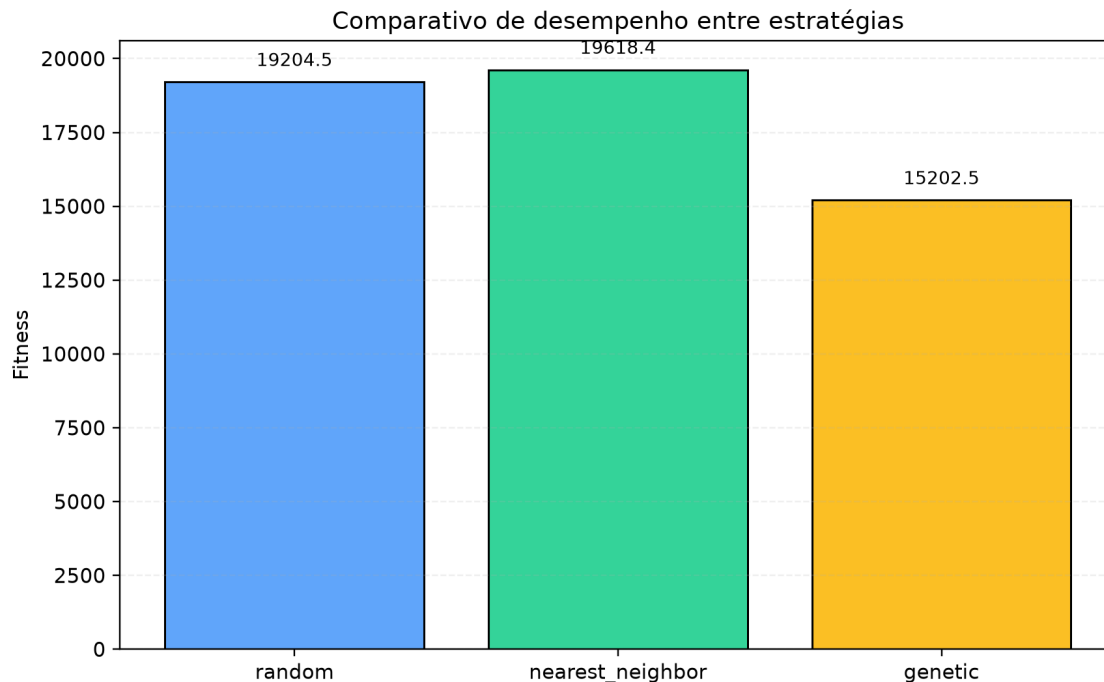
6. Comparativo de desempenho e análise das rotas

A validade da abordagem foi verificada por meio de benchmarks comparando o algoritmo genético com outras estratégias, como rota aleatória e vizinho mais próximo (conforme imagem abaixo).

Os resultados mostraram que a solução evolutiva apresentou melhor desempenho em cenários com múltiplas restrições, alcançando fitness superior e maior capacidade de equilibrar entregas entre veículos. Além disso, a análise das rotas permitiu verificar a distribuição das entregas, a utilização de capacidade e autonomia, e a presença de prioridades críticas em posições adequadas.

Também as visualizações geradas pela interface em Streamlit e pela execução local em Pygame facilitaram a interpretação dos resultados e ajudaram na identificação de gargalos operacionais e pontos de melhoria.





7. Desafios, limitações e melhorias futuras

Durante o desenvolvimento, os principais desafios foram a criação de uma representação de rota compatível com múltiplos veículos, a manutenção de soluções viáveis e o equilíbrio entre distância e penalização por restrições.

Cenários com alta demanda, baixa autonomia ou grande número de entregas críticas exigiram atenção especial. Para lidar com isso, o projeto incorporou mecanismos de reparo, avaliação por penalidades e tratamento explícito de entregas críticas.

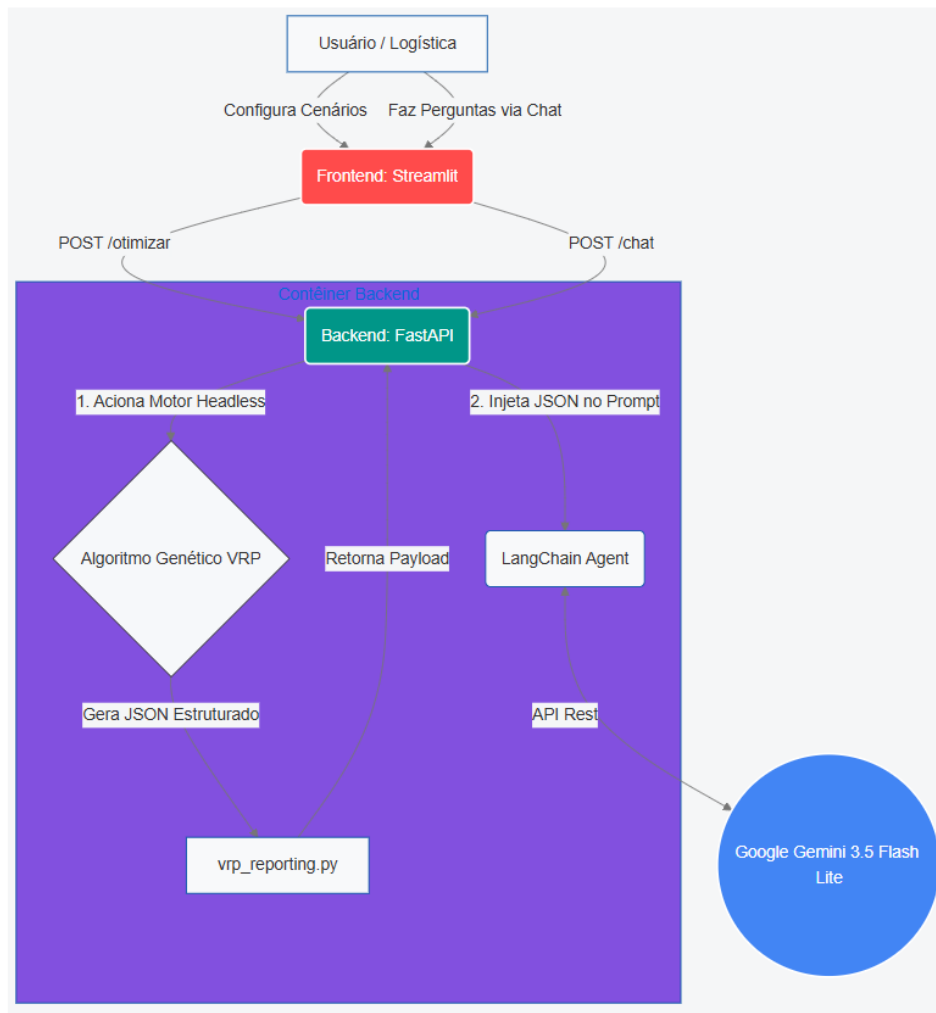
Como limitações, a solução ainda depende de cenários bem definidos e de dados estruturados para a geração de relatórios por IA. Como melhorias futuras, o sistema pode evoluir para otimização multiobjetivo mais avançada, integração com dados em tempo real e visualização geográfica mais detalhada das rotas.

8. Arquitetura da solução

A arquitetura foi organizada em camadas para separar interface, processamento e inteligência artificial. O usuário interage com a interface em Streamlit, que envia requisições para o backend em FastAPI.

O backend executa o algoritmo genético, gera um relatório estruturado em JSON e encaminha os resultados para o agente de linguagem baseado em LangChain. O modelo Gemini interpreta o contexto operacional e produz instruções e relatórios em linguagem natural.

Esse fluxo conecta a entrada do usuário, a otimização da rota, a análise de restrições, a geração de resposta e a apresentação dos resultados em uma sequência lógica e funcional.



9. Conclusão

O projeto demonstrou que é possível combinar otimização matemática e inteligência artificial para resolver um problema prático e relevante em logística hospitalar.

A implementação do algoritmo genético foi fundamental para explorar combinações de rotas de forma eficiente, enquanto as estratégias de penalização e reparo permitiram lidar com restrições reais do problema. A integração com LLMs ampliou a utilidade do sistema ao transformar resultados técnicos em orientações operacionais e relatórios acessíveis.

O conjunto desenvolvido oferece uma solução funcional, técnica e útil para apoio à decisão logística, com potencial de evolução para cenários mais complexos e dinâmicos.